



Tecnológico de Monterrey

Evidencia 1 - Actividad Integral de Conceptos Básicos y Algoritmos Fundamentales

**Programación de estructuras de datos y algoritmos fundamentales
| TC1031.607**

David Alonso Cantú Delgado.

Regina Mugica Limon | A00843542

*20 de septiembre del 2026
Monterrey, Nuevo León*

Reflexión

En esta evidencia pude entender mucho mejor para qué sirven realmente los algoritmos de ordenamiento y búsqueda. Antes los había visto principalmente en ejercicios de clase y sabía cómo funcionaban, pero no había visto tanto la diferencia que puede hacer escoger un algoritmo u otro cuando ya tienes muchos datos.

En este caso trabajamos con archivos de 6,818 registros, entonces primero era necesario ordenarlos por fecha y hora para después poder buscar información de una manera más fácil. Esto me ayudó a entender por qué es importante tener los datos ordenados, ya que una vez que los registros están en orden se puede utilizar una búsqueda binaria y no es necesario empezar buscando uno por uno desde el inicio.

También entendí mejor que la búsqueda binaria no solamente sirve para buscar un dato exacto. En este programa la usamos para encontrar el inicio y el final de un rango de fechas, lo cual fue algo diferente a los ejercicios que había hecho anteriormente.

Algo que me llamó mucho la atención fue que no todos los algoritmos se comportaron igual con los dos archivos. Yo pensaba que si un archivo estaba casi ordenado, en general los algoritmos iban a ser más rápidos, pero con las pruebas pude ver que depende mucho de cómo funciona cada uno.

Por ejemplo, Bubble Sort fue mucho más rápido con el archivo casi ordenado. Con el archivo desordenado tardó 1,432,364,292 ns y con el casi ordenado solamente 10,759,542 ns. Algo parecido pasó con Insertion Sort, que pasó de 862,622,833 ns a 5,362,208 ns. Esto me ayudó a ver de una forma más clara cómo estos algoritmos sí pueden aprovechar que los datos ya tengan cierto orden.

Selection Sort fue muy diferente porque prácticamente tardó lo mismo en los dos archivos. Obtuvo 679,058,125 ns con el archivo desordenado y 665,145,333 ns con el casi ordenado. Esto tiene sentido porque de todas formas tiene que seguir recorriendo los elementos para encontrar el menor.

Merge Sort también fue bastante constante. Sus tiempos fueron 35,419,917 ns y 31,734,459 ns. En este caso los resultados tienen sentido con lo que vimos de su complejidad $O(n \log n)$, ya que no depende tanto de que los datos estén previamente ordenados. Estos son los mismos resultados que obtuve durante las pruebas de los siete algoritmos.

Antes de correr cada algoritmo hice una predicción sobre si esperaba que fuera rápido o lento dependiendo del algoritmo y de cuál archivo estaba utilizando. La mayoría de mis predicciones sí coincidieron con lo que pasó, pero hubo resultados que me parecieron más interesantes que otros.

El que más me llamó la atención fue Quick Sort. En el archivo desordenado tardó solamente 14,111,625 ns, pero en el archivo casi ordenado tardó 212,582,959 ns. Antes de correrlo yo había predicho que podía ser más lento porque en mi implementación se utiliza el último elemento como pivote.

Al ver el resultado entendí mejor por qué la manera en la que se escoge el pivote importa. Como el archivo ya estaba casi ordenado, utilizar el último elemento podía generar divisiones que no estuvieran bien balanceadas y hacer que Quick Sort se acercara más a su peor caso $O(n^2)$. Para mí esta fue de las partes más útiles de la actividad porque pude ver algo que habíamos visto en teoría reflejado directamente en mis resultados.

Durante la actividad utilicé inteligencia artificial como apoyo cuando tenía dudas sobre partes específicas del programa o cuando quería revisar si algo que había hecho tenía sentido. Algunas de las cosas que consulté fueron cómo manejar los límites de la búsqueda binaria, cómo tratar registros que tuvieran exactamente el mismo timestamp y por qué algunos algoritmos podían comportarse diferente con un archivo casi ordenado.

Algunos de los prompts que utilicé fueron: “¿Cómo puedo hacer la búsqueda por rango utilizando búsqueda binaria?”, “¿Qué pasa si existen dos registros con exactamente el mismo timestamp?” y “¿Por qué Quick Sort puede ser más lento si el archivo está casi ordenado?”. Estas preguntas me ayudaron principalmente a entender y revisar partes específicas de mi programa.

También hubo un momento donde me di cuenta de que no podía simplemente confiar en una sugerencia porque el código corriera. Al principio, `range607.txt` solamente se abría para escribir cuando la búsqueda encontraba registros. Cuando hice la prueba de un rango vacío me di cuenta de que eso causaba un problema, porque el archivo podía quedarse con los resultados de la búsqueda anterior aunque la nueva búsqueda no hubiera encontrado nada.

Para corregirlo cambié el lugar donde se abría `range607.txt`, de manera que también se limpiara cuando no hubiera resultados. Después volví a hacer la prueba con un rango vacío para comprobar que ahora sí funcionara correctamente. Esta prueba también la incluí en mis evidencias.

Creo que esta parte fue importante porque me hizo ver que la inteligencia artificial sí me puede ayudar cuando estoy programando, pero no significa que todo lo que me dé va a funcionar correctamente para mi programa. Al final yo tengo que entender qué está haciendo el código, probarlo y decidir si realmente funciona para lo que necesito.

En general, esta evidencia me ayudó a entender mejor los algoritmos porque ya no solamente los vi por separado como ejercicios de clase. Al compararlos con los mismos datos pude ver que la

complejidad sí importa, pero también importa cómo vienen organizados los datos y cómo está implementado cada algoritmo.

También entendí mucho mejor la búsqueda binaria y la importancia de pensar en casos que normalmente no consideraría al principio, como un rango donde no existen registros o dos registros que tienen exactamente la misma fecha y hora. Las pruebas me ayudaron a encontrar este tipo de detalles y a comprobar que el programa funcionara no solamente en el caso más sencillo.

Creo que lo que más me llevo de esta actividad es entender que no existe un algoritmo que siempre sea el mejor para todo. Dependiendo de los datos y de la manera en la que está implementado, su comportamiento puede cambiar bastante. También aprendí que probar el programa con diferentes casos es igual de importante que lograr que compile.